



TÉCNICAS DE PROGRAMAÇÃO II

PROFº LUIZ CLÁUDIO

ENCAPSULAMENTO UTILIZANDO PROPRIEDADES

Getters e Setters: usando *Properties*

```
class Pessoa:
    def __init__(self, nome, idade):
        self.__nome = nome
        self.idade = idade
```

```
@property # get
def nome(self): return
    self.__nome
```

```
@nome.setter #set
def nome(self, nome):
    self.__nome = nome
```

Ao utilizar properties , o encapsulamento dos métodos e visto como se fosse um método



ENCAPSULAMENTO UTILIZANDO PROPRIEDADES – CRIANDO ATRAVÉS DO PROP DE FORMA AUTOMÁTICA VISUAL STUDIO CODE

Pode criar snippets de código que geram automaticamente @property e setter

Como criar no VS Code:

1- Pressione Ctrl + Shift + P

2- Digite:

Snippets: Configure User Snippets

•Escolha python.json

•Adicione: no arquivo ->>>>>>>>>>

Agora basta digitar:

Prop e pressionar TAB.

```
@property # get
def nome(self): return
    self.__nome

@nome.setter #set
def nome(self, nome):
    self.__nome = nome
```

Será criado automaticamente e as properties, basta alterar pelo nome de cada atributo

```
{
  "Property Getter Setter": {
    "prefix": "prop",
    "body": [
      "@property",
      "def ${l:nome}(self):",
      "    return self._${l:nome}",
      "",
      "@${l:nome}.setter",
      "def ${l:nome}(self, valor):",
      "    self._${l:nome} = valor"
    ],
    "description": "Criar property getter setter"
  }
}
```

Coloque isto no arquivo python.json



ENCAPSULAMENTO UTILIZANDO PROPRIEDADES

Getters e Setters: usando *Properties* classe Principal

```
if __name__ == "__main__":
    pessoa = Pessoa("Ana", 20, True)
```

```
# Utilizando properties
pessoa.nome = "José"
print(pessoa.nome)
```

Ao utilizar properties , o encapsulamento dos métodos e visto como se fosse um método, assim so colocar o nome da classe seguido de ponto e nome do atributo (. nome do atributo)



ENCAPSULAMENTO

EXEMPLO PROPRIEDADES CLASSE FORNECEDORES

```
class Fornecedores:

    def __init__(self):
        self.__nomeFornecedor = ""
        self.__nomeProduto = ""
        self.__descricaoProduto = ""

    # PROPERTY nomeFornecedor
    @property
    def nomeFornecedor(self):
        return self.__nomeFornecedor

    @nomeFornecedor.setter
    def nomeFornecedor(self, valor):
        self.__nomeFornecedor = valor

    # PROPERTY nomeProduto
    @property
    def nomeProduto(self):
        return self.__nomeProduto

    @nomeProduto.setter
    def nomeProduto(self, valor):
        self.__nomeProduto = valor
```

```
# PROPERTY descricaoProduto
@property
def descricaoProduto(self):
    return self.__descricaoProduto

@descricaoProduto.setter
def descricaoProduto(self, valor):
    self.__descricaoProduto = valor
```

Os atributos são criados com a anotação @property



ENCAPSULAMENTO

EXEMPLO PROPRIEDADES CLASSE FORNECEDORES

```
# Método cadastrar
def cadastrarFornecedor(self):

    print("\n=== Cadastro de Fornecedor ===")

    self.nomeFornecedor = input("Digite o nome do fornecedor: ")
    self.nomeProduto = input("Digite o nome do produto: ")
    self.descricaoProduto = input("Digite a descrição do produto: ")

    print("Fornecedor cadastrado com sucesso!")

# Método listar
def listarFornecedor(self):

    print("\n=== Dados do Fornecedor ===")

    print("Fornecedor:", self.nomeFornecedor)
    print("Produto:", self.nomeProduto)
    print("Descrição:", self.descricaoProduto)
```

Ao utilizar properties , o encapsulamento dos métodos e visto como se fosse um método, assim so colocar o nome do construtor self ,seguido de ponto e nome do atributo (. nome do atributo)



ENCAPSULAMENTO

EXEMPLO PROPRIEDADES CLASSE FORNECEDORES

```
from fornecedor import Fornecedores

class Principal:

    @staticmethod
    def main():
        fornecedor = Fornecedores()
        fornecedor.cadastrarFornecedor()
        fornecedor.listarFornecedor()

# equivalente ao main de Java
if __name__ == "__main__":
    Principal.main()
```

Cria o objeto fornecedor e chama os métodos da classe Fornecedores

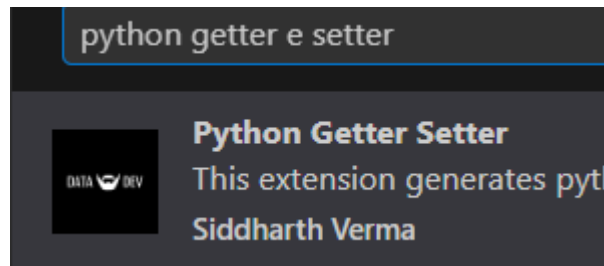


ENCAPSULAMENTO UTILIZANDO PROPRIEDADES – CRIANDO ENCAPSULAMENTO DE FORMA AUTOMÁTICA COM TODOS OS ATRIBUTOS.

Pode criar snippets de código que geram automaticamente @property e setter

Para gerar métodos get e set automaticamente:

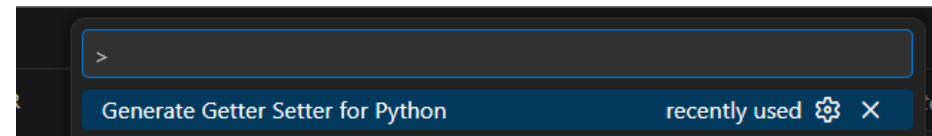
1.Instale a extensão: Vá até a aba de extensões (Ctrl+Shift+X), pesquise por "**Python Getter Setter**" (por Siddharth Verma) e instale.



Digite python getter e setter e instale a extensão

2.Selecione o código: No seu arquivo .py, selecione a variável ou as variáveis (atributos) dentro da classe que você deseja encapsular.

3.Abra a Paleta de Comandos: Pressione Ctrl+Shift+P.



4.Execute o comando: Digite "Generate Getter Setter for Python" e pressione Enter.

5.Resultado: A extensão gerará automaticamente os métodos ou decoradores @property para o atributo selecionado



ENCAPSULAMENTO UTILIZANDO PROPRIEDADES – CRIANDO ENCAPSULAMENTO DE FORMA AUTOMATICA COM TODOS OS ATRIBUTOS.

```
class Fornecedores:

    def __init__(self):
        self.__nomeFornecedor = ""
        self.__nomeProduto = ""
        self.__descricaoProduto = ""

    @property
    def _nomeFornecedor(self):
        return self.__nomeFornecedor

    @_nomeFornecedor.setter
    def _nomeFornecedor(self, value):
        self.__nomeFornecedor = value

    @property
    def _nomeProduto(self):
        return self.__nomeProduto

    @_nomeProduto.setter
    def _nomeProduto(self, value):
        self.__nomeProduto = value

    @property
    def _descricaoProduto(self):
        return self.__descricaoProduto

    @_descricaoProduto.setter
    def _descricaoProduto(self, value):
        self.__descricaoProduto = value
```

Os getter e setters são criados automaticamente após selecionar os atributos e executar o Generate Getter e Setter



ENCAPSULAMENTO

EXEMPLO PROPRIEDADES CLASSE FORNECEDORES

```
# Método cadastrar
def cadastrarFornecedor(self):

    print("\n=== Cadastro de Fornecedor ===")

    self.__nomeFornecedor = input("Digite o nome do fornecedor: ")
    self.__nomeProduto = input("Digite o nome do produto: ")
    self.__descricaoProduto = input("Digite a descrição do produto: ")

    print("Fornecedor cadastrado com sucesso!")
```

```
# Método listar
def listarFornecedor(self):

    print("\n=== Dados do Fornecedor ===")

    print("Fornecedor:", self.__nomeFornecedor)
    print("Produto:", self.__nomeProduto)
    print("Descrição:", self.__descricaoProduto)
```

Neste caso ao utilizar properties , o encapsulamento dos atributos é só colocar os atributos como private o nome do construtor self ,seguido de ponto e __nome do atributo (.__nome do atributo)





OBRIGADO

LUIZ.BARRETO@CPS.SP.GOV.BR